

This is a walkthrough of the multi-agent system I built to run order processing for Beaver's Choice Paper Company. A customer email comes in asking for paper products, and the system has to read it, check what we have, set a price, work out delivery timing, place any needed orders, and write back a reply, all without a person reading the email first. The clearest way to explain this system is to follow the data itself: which tool writes which fact and where that fact ends up. That's the order I'll go in here, following the same shape as the system diagram.

Every request runs with a single object called `SharedState`. It starts almost empty and fills up as the request moves along. Every tool in the system, no matter which agent owns it, follows the same pattern: it does one piece of work, then writes its result into a specific named field on `SharedState`. No agent is allowed to just state a fact and have it count. A number only becomes real once a tool has actually fetched it from the database or computed it from a fixed rule and written it down.

That single rule is what makes the rest of this system traceable. At any point you can look at `SharedState` and see exactly which tool produced each fact.

The orchestration agent and its tools

The orchestration agent is the only one that sees the customer's email directly, and it's the one that produces the final reply. It has seven tools, and each one writes into a different part of `SharedState`.

`record_email_details` reads the raw email and writes the items requested, the request date, and the quantities into `SharedState`. This is always the first real step, because almost everything downstream depends on knowing what was asked for and when.

`get_delivery_address` writes a `shipping_address` into `SharedState`. There's no real customer database behind this in a fictional company, so this tool fills that gap directly instead of leaving it open.

`generate_financial_report_dict` builds a financial report and writes it into `SharedState.financial_report`. This gives the orchestrator a snapshot of cash and inventory as of the request date, before any decisions get made.

`call_inventory_manager` hands a question to the inventory agent and gets back a `WorkerOutput`, which it writes into `SharedState` as stock facts, the inventory numbers relevant to this specific request.

`determine_stock_needs` also goes to the inventory agent, but with a fixed question: given what was requested and what we have, how much more do we need to order. The result is written into `SharedState` as `StockNeeds`.

call_quoting_manager hands a question to the quoting agent and writes the result into SharedState as history and cash, meaning past quote context and, if relevant, our current cash position.

call_ordering_manager hands a question to the ordering agent and writes the result into SharedState as ETAs and transactions, meaning delivery timing and any orders that were actually placed.

Once all seven of these have run, the orchestrator has every fact it needs sitting in SharedState, and it produces the final output: the customer-facing reply.

The three worker agents

Below the orchestration agent sit three worker agents, each with its own small set of tools. None of them ever talk to the customer directly. They only ever respond to the orchestrator's *call_X_manager* tools, and they only ever write into the same SharedState object.

The inventory agent has two tools. *get_inventory_for_date* pulls a full snapshot of every item in stock and writes it into *inventory_levels_of_all_products*. *get_stock_level_item* checks one specific item and writes the result into *stock_level_specific_items*. The first is for broad "What do you have?" questions; the second is for checking a named item the customer actually asked about.

The quoting agent also has two tools. *get_cash_balance_value* checks our cash position as of a given date and writes it into *cash_balance*. *search_quote_history_retrieve* looks up past quotes for similar items and writes them into *quote_history*. Together these give the quoting agent enough to recommend a price grounded in precedent and to flag whether we can actually afford a new supplier order if one is needed.

The ordering agent has three tools, and it reads more from SharedState than it writes, since it depends on stock levels, delivery estimates, prices, and the request date all being in place first. *get_supplier_delivery_date_estimate* works out how long it will take stock to arrive from our supplier and writes it into *supplier_delivery_date*. *get_customer_delivery_date_estimate* works out how long it will take an order to reach the customer and writes it into *customer_delivery_date*. *create_transaction_record* is the one tool in the entire system that actually changes the database, recording a stock order or a sale, and it writes the result into *orders_completed*.

Of the three worker agents, the ordering agent ends up doing the most. It both judges whether an order is actually deliverable in time and writes the financial record once that judgment is made. Looking back, this is the one place I'd split into two agents if I had more to work with: one that only works out delivery timing and a separate one that only records transactions once a

decision has already been made elsewhere. As it stands, one agent owning both the judgment and the record of that judgment is a single point of failure I'd want to remove given the chance.

The data extraction agent

This agent doesn't appear as its own box with tools in the diagram because it has no tools of its own. It sits behind *record_email_details*, and its only job is reading the customer's email closely enough to identify the goals of the request, the request date, the items named, a matched version of those item names against our actual catalog, the quantities, and any delivery date the customer asked for. Everything downstream depends on getting this step right, since a wrong match here means the inventory and ordering agents end up checking and ordering the wrong item entirely.

The database underneath all of this

Every one of the tools above is backed by a plain SQLite database with three tables: **transactions**, **quotes**, and **inventory**. The model is kept thin on purpose. There's no separate idea of an order anywhere in the database. Every supplier purchase and every customer sale is just one row in transactions, tagged as a stock order or a sale. The stock level for any item on any date is the total of stock orders minus the total of sales up to that date. Cash balance on any date is the total of sales minus the total of stock order costs up to that date. A financial report combines that cash figure with the value of everything currently in inventory and a short list of best-selling items.

Every one of these calculations filters by date, so the system can answer a question about the past exactly the same way it answers a question about right now. That matters because the system replays old customer requests in date order during testing, and a calculation from three weeks ago should never accidentally use today's numbers.

Two smaller pieces round this out. Delivery lead times for both the supplier and the customer come from a fixed lookup based on order size, written directly into code instead of left for a model to guess. Past quotes are searchable by keyword, which is what gives the quoting agent something real to compare new requests against.

Why the system is built this way

The framework underneath all of this is **pydantic_ai**, chosen mainly because it forces every agent's output into a fixed shape. If a worker agent doesn't return something in the right shape, the framework retries instead of letting a broken answer move forward. That's a big part of why the data flow described above stays reliable: nothing advances until it matches the shape the next step expects.

Splitting the work across five agents instead of building one large agent came directly from the fact that each step depends on the one before it. The ordering agent can't judge feasibility without knowing what's in stock, and the quoting agent can't set a price without knowing which items were matched. None of these steps stand alone, which is the entire reason something has to sit above them and pass the right fact to the right specialist at the right time.

Which agent owned which tool changed a fair amount during testing. Early versions had agents calling the same tool repeatedly with slightly different inputs, trying to double-check answers they already had, which burned through the request budget without learning anything new. The fix was to give each agent a strict instruction to check `SharedState` first, call each tool only once per unique input, and stop as soon as the goal was answered. Even after that change, I still had to raise how many retries and tool calls each request was allowed, since some of the harder customer requests genuinely need several rounds of back and forth to resolve properly.

What testing against real requests showed

Running this against a real set of historical customer requests, the data flow held up the way I wanted. I didn't see one agent's numbers contradicting another's anywhere I checked, which was the main risk going in. The separate `determine_stock_needs` step, instead of folding that calculation into a broader inventory question, made that one number consistent and easy to check by hand. Logging every tool call and writing each run's reasoning to its own file made it easy to trace a strange-looking answer back to the exact fact that caused it.

Two issues stood out as worth fixing properly. Left alone, the model wanted to use its own sense of today instead of the date the customer actually wrote the request, which is the right instinct most of the time but wrong here, where every calculation has to be anchored to the request date. I had to make that rule explicit everywhere a date gets used. The system also kept wanting to pause and ask the customer for confirmation before finishing an order, and when a delivery looked tight, it would try to invent solutions that don't exist in this business, like expedited shipping. The fix for both was the same: stop leaving room for the model to fill a gap on its own, and instead close that gap directly, either with a firm rule in the prompt or, in the case of missing shipping addresses, a tool that generates one outright.

Where I'd take this next

Even though I'm happy with how this turned out, there's a fair amount I'd still change given more time and a bigger budget for testing, since iterating on this used up a lot of credits already.

The first thing I'd fix is `create_transaction_record` itself. Right now it's one tool shared by the ordering agent for both stock orders and customer sales, which means every transaction has to be routed through that single agent even when the inventory agent already has everything it needs to place a restock on its own. Splitting it into two separate tools, one for stock orders that

the inventory agent can call directly, and one for sales that stays with the ordering agent would cut down on how much the agents need to hand work back and forth just to write a single row to the database.

The second thing is that quotes given to customers aren't actually fed back into the system anywhere. They get saved for evaluation purposes, but *search_quote_history_retrieve* only ever searches the original quotes file the system started with, not anything it has quoted since. In a real version of this, I'd want every quote the system generates to get written somewhere that future requests can search, either as its own record in the database alongside transactions or as a simple log of the prompt and the resulting quote that the search tool also checks. Without that, the system never actually learns from its own pricing history as it runs.

I'd also look harder at how much of this has to happen in order. Right now most of the steps run one after another because each one depends on something the last one produced, but a few of them don't really need to wait on each other. Pulling the financial report and the delivery address at the same time as the inventory check, for example, doesn't depend on stock levels at all, and running those in parallel instead of in sequence would cut down on how long a single request takes to resolve.

Last, even with the stop conditions and idempotency rules I put in place, I still noticed tool calls that didn't quite make sense, the same tool getting called again with a slightly different date or quantity, as if the agent were trying out small variations rather than reusing what it already had. I never fully solved this. It's a real cost, both in time and in the credits each of those extra calls uses, and I think there's more room to tighten the rules around exactly when a tool is allowed to be called again with different inputs.